

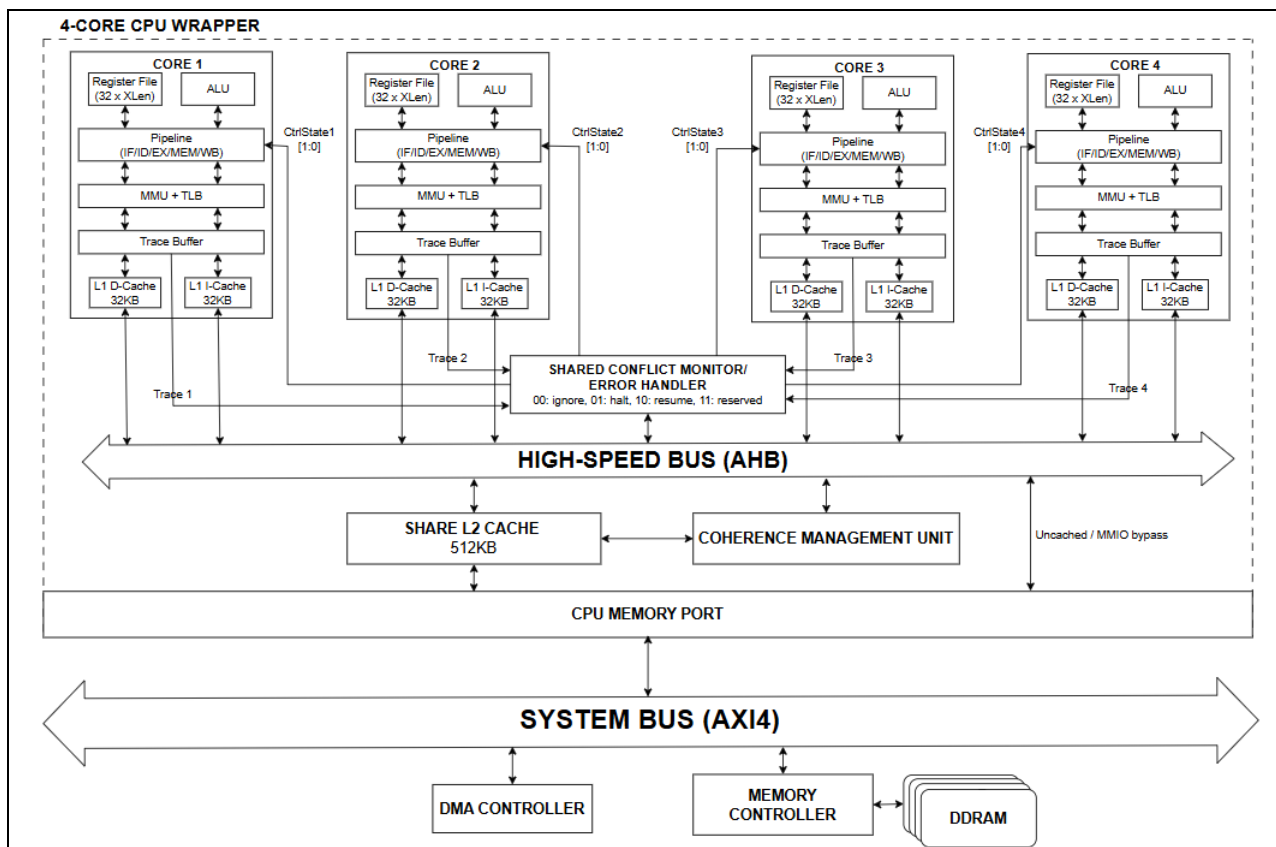
ĐỀ CƯƠNG CHI TIẾT

TÊN ĐỀ TÀI TIẾNG VIỆT: THIẾT KẾ BỘ VI XỬ LÝ 4 LỖI RISC-V DỰA TRÊN KIẾN TRÚC TẬP LỆNH RV32IMA
TÊN ĐỀ TÀI TIẾNG ANH: DESIGN OF A QUAD-CORE RISC-V MICROPROCESSOR BASED ON THE RV32IMA INSTRUCTION SET ARCHITECTURE
Cán bộ hướng dẫn Họ và tên: TS. Nguyễn Minh Sơn Nơi công tác: Trường Đại học Công nghệ Thông tin ĐHQG TP. HCM
Thời gian thực hiện: Từ ngày 07/09/2026 đến ngày 27/12/2026
Sinh viên thực hiện: Vũ Thành Lam - 23520840 Võ Lê Phước Lâm - 23520838
Nội dung đề tài: Tổng quan đề tài Trong nhiều hệ thống nhúng hiện đại, khối xử lý trung tâm không còn chỉ đảm nhiệm một tác vụ đơn lẻ như một vi điều khiển (microcontroller) truyền thống (ví dụ: một MCU chuyên trách điều khiển động cơ - motor control), mà cần đóng vai trò một bộ xử lý đa dụng (general-purpose processor), đủ mạnh để đồng thời giao tiếp, xử lý dữ liệu và điều phối nhiều bộ điều khiển/cảm biến khác nhau trong hệ thống. Một ví dụ tiêu biểu là khối Flight Control (FC) trên phương tiện bay không người lái (UAV): khối này phải liên tục thu thập dữ liệu từ nhiều cảm biến (IMU, GPS, cảm biến độ cao...), chạy đồng thời các thuật toán ước lượng trạng thái, điều khiển bay và xử lý giao tiếp với các module ngoại vi khác, trong khi vẫn đảm bảo thời gian đáp ứng thực. Hiệu năng xử lý đơn lõi không còn đáp ứng đủ cho các chức năng tự động hóa nâng cao dựa trên các thuật toán tính toán phức tạp như học máy (Machine Learning) hay xử lý ảnh (vision-based processing) trên các thể hệ máy tính bay (flight computer) tiếp theo [1]. Đây chính là động lực trực tiếp cho việc sử dụng một bộ xử lý đa lõi (multi-core processor) trong các hệ thống loại này. Trên thế giới, hướng tiếp cận dùng bộ xử lý đa lõi cho các hệ thống Flight Control/hệ thống điều khiển bay đã được nhiều nền tảng thương mại và nghiên cứu áp dụng. Trong đó, kiến trúc ARM chiếm ưu thế với các thiết kế như bộ xử lý DAHLIA (dựa trên lõi ARM Cortex-R52, do Liên minh châu Âu tài trợ phát triển) hay hệ thống đa lõi do Boeing phát triển trong chương trình Next-generation High Performance Spaceflight Computing (HPSC) của NASA, sử dụng 8 lõi ARM Cortex-A53 kết hợp 2 lõi Cortex-R52 [1]. Tuy nhiên, kiến trúc ARM là kiến trúc tập

lệnh sở hữu độc quyền, đòi hỏi chi phí bản quyền lõi (core licensing) và hạn chế khả năng tùy biến ở mức vi kiến trúc. Đáng chú ý, cùng trong chương trình HPSC nêu trên, SiFive và Microchip cũng đã được trao hợp đồng phát triển một bộ xử lý đa lõi dựa trên RISC-V cho ứng dụng không gian [1], cho thấy RISC-V đang dần được công nhận là một hướng tiếp cận khả thi ngay trong chính lĩnh vực hàng không vũ trụ. RISC-V (Reduced Instruction Set Computer - V), ra đời từ năm 2011, là kiến trúc tập lệnh mã nguồn mở, mô-đun hóa và cho phép tùy biến sâu ở cấp độ RTL, phù hợp với mục tiêu nghiên cứu, học thuật và khả năng mở rộng của đề tài. Đây là lý do nhóm lựa chọn hướng thiết kế một hệ thống đa lõi dựa trên RISC-V (cụ thể là RV32IMA) thay vì sử dụng lõi ARM có sẵn.

Hướng nghiên cứu về hệ thống đa lõi dựa trên RISC-V nói chung cũng đã được nhiều nhóm nghiên cứu quan tâm. Có thể kể đến công trình "Hardware Software Co-design for Multi-threaded Computation on RISC-V-based Multicore System" [2], tập trung vào cơ chế lập lịch tác vụ (task scheduling) để khai thác song song hóa trên hệ đa lõi RISC-V, và "A Low-Overhead Reconfigurable RISC-V Quad-Core Processor Architecture for Fault-Tolerant Applications" [3], trình bày một kiến trúc 4 lõi RV32IM có khả năng cấu hình lại giữa chế độ thường và chế độ chịu lỗi (fault-tolerant). Ở trong nước, tại Khoa Kỹ thuật Máy tính, Trường Đại học Công nghệ Thông tin, hướng thiết kế bộ xử lý RISC-V trên FPGA đã được triển khai qua nhiều khóa luận và phát triển dần qua từng năm. Gần nhất, khóa luận "Thiết kế bộ vi xử lý 2 lõi RISC-V dựa trên kiến trúc tập lệnh RV32IMFA" [4] đã hiện thực thành công một hệ thống 2 lõi RV32IMFA với cơ chế phân xử truy cập bộ nhớ dùng chung và hỗ trợ lệnh nguyên tử, kiểm chứng trên kit FPGA KV260; tuy nhiên hệ thống chưa tích hợp bộ nhớ đệm (cache), chưa có cơ chế duy trì tính nhất quán dữ liệu (cache coherence) và chưa có cơ chế quản lý bộ nhớ ảo. Song song đó, khóa luận "Thiết kế bộ điều khiển đồng bộ cache cho các hệ thống 2 lõi dựa trên kiến trúc tập lệnh RISC-V RV32IA" [5] đã hiện thực một hệ thống 2 lõi với cache L1/L2 theo cấu trúc 4-way set-associative, kết nối qua chuẩn AXI4 Full, cùng cơ chế duy trì tính nhất quán bộ nhớ đệm theo giao thức MOESI Snoopy (5 trạng thái) và hỗ trợ đầy đủ cho RV32A, kiểm chứng trên kit FPGA Virtex-7 VC707; tuy nhiên hệ thống này cũng dừng ở quy mô 2 lõi và chưa tích hợp cơ chế quản lý bộ nhớ ảo (MMU/TLB).

Qua khảo sát trên, có thể thấy: các công trình quốc tế [2][3] tập trung vào lập lịch phần mềm và khả năng chịu lỗi trên hệ đa lõi RISC-V nhưng chưa đi sâu vào tổ chức bộ nhớ đệm nhiều cấp; trong khi đó, công trình gần nhất tại khoa mới dừng ở quy mô 2 lõi, và dù đã có một công trình hiện thực cache/coherence [5], cả hai công trình liền kề nhất [4][5] đều chưa tích hợp cơ chế quản lý bộ nhớ ảo cho từng lõi. Trên cơ sở nhu cầu ứng dụng thực tế đã nêu và khoảng trống nghiên cứu này, nhóm đề xuất thiết kế một hệ thống bộ vi xử lý RISC-V 4 lõi dựa trên tập lệnh RV32IMA (I - số nguyên cơ bản, M - nhân/chia, A - thao tác nguyên tử), có tổ chức bộ nhớ đệm riêng từng lõi kết hợp cache dùng chung, cơ chế duy trì tính nhất quán dữ liệu (cache coherence) hoạt động ổn định trong môi trường nhiều lõi truy cập đồng thời, cùng bộ quản lý bộ nhớ (MMU/TLB) hỗ trợ bộ nhớ ảo cho từng lõi.



Mục tiêu của đề tài

Nghiên cứu, thiết kế và hiện thực thành công một hệ thống bộ vi xử lý RISC-V 4 lõi dựa trên kiến trúc tập lệnh RV32IMA bằng ngôn ngữ mô tả phần cứng SystemVerilog, mô phỏng và kiểm chứng trên công cụ Vivado, với hai mục tiêu chính như sau:

Mục tiêu 1 - Thiết kế kiến trúc CPU 4 lõi (multi-core) tích hợp bộ nhớ cache: thiết kế kiến trúc bộ xử lý 4 lõi RV32IMA tích hợp bộ nhớ cache (L1/L2) bằng SystemVerilog, bao gồm: (i) thiết kế IP cache L1 (lệnh/dữ liệu, riêng từng lõi) và cache L2 (dùng chung); (ii) thiết kế bộ điều khiển cache với thuật toán cấp phát/thay thế set-associative và giao thức duy trì tính nhất quán bộ nhớ đệm; (iii) thiết kế MMU cho từng lõi, gồm TLB và PTW hỗ trợ dịch địa chỉ Sv32; (iv) thiết kế kiến trúc liên kết hai tầng, trong đó AHB-Lite nối các lõi/cache L1 với cache L2, còn AXI4 nối L2, DMA, DRAM và ngoại vi; và (v) thiết kế DMA phục vụ trao đổi dữ liệu khối với thiết bị lưu trữ ngoài.

Mục tiêu 2 - Mô phỏng, đánh giá thiết kế và hiện thực trên FPGA: kiểm chứng chức năng của toàn bộ thiết kế bằng mô phỏng RTL, đánh giá tài nguyên phần cứng và năng lượng tiêu thụ khi tổng hợp bằng Vivado, và hiện thực, kiểm tra hoạt động thực tế trên kit FPGA với kịch bản kiểm tra phản ánh đúng cách một bộ xử lý (processor) thật hoạt động, đồng thời dùng làm cơ sở đánh giá hiệu năng/tốc độ xử lý của hệ thống quad-core (xem chi tiết ở Nội dung 4 và Demo dự kiến).

Phương pháp thực hiện

Đề tài được triển khai theo hướng tiếp cận thiết kế phần cứng tăng dần độ phức tạp, đi từ lõi xử lý đơn đến hệ thống nhiều lõi, cụ thể qua các bước sau:

- Bước 1 - Khảo sát và nghiên cứu lý thuyết: khảo sát nhu cầu ứng dụng thực tế và các hướng nghiên cứu đa lõi RISC-V; tìm hiểu RV32I/M/A, hệ thống đa lõi, cache nhiều cấp và coherence; nghiên cứu MMU/TLB với cơ chế dịch địa chỉ Sv32; đồng thời phân tích AHB-Lite và AXI4 theo vai trò khác nhau trong hệ thống. AHB-Lite được dùng ở miền truy cập

từ lỗi/cache L1 đến L2 vì giao thức đơn giản, phù hợp phân xử nhiều master; AXI4 được dùng ở miền hệ thống L2-DMA-DRAM-ngoại vi nhờ hỗ trợ nhiều kênh độc lập và truyền burst. Hai chuẩn được sử dụng bổ sung theo kiến trúc phân tầng, không phải hai phương án thay thế lẫn nhau.

- Bước 2 - Thiết kế và hiện thực nền tảng trong tháng 9-10: hiện thực lõi RV32IMA pipeline 5 tầng; hoàn thiện MMU/TLB/PTW và kiểm chứng hit, miss, refill, page walk, page fault; xây dựng AHB-Lite interconnect phía core-L2 và AXI4 system bus phía L2-DMA-DRAM/ngoại vi; sau đó tích hợp lõi, MMU, cache L1 và giao tiếp AHB-Lite thành một khối đơn lõi ổn định trước khi nhân bốn.
- Bước 3 - Mở rộng thành hệ thống 4 lõi: nhân bản khối lõi đơn đã có MMU/TLB và cache L1 thành 4 lõi, đấu nối qua AHB-Lite đến cache L2 dùng chung; kết nối miền L2 với AXI4 system bus thông qua giao diện/bridge xác định rõ, rồi tích hợp coherence, DMA và bộ điều khiển SD/SPI. Kiểm chứng riêng handshake, arbitration, back-pressure và burst transfer của từng miền bus trước khi chạy các kịch bản chia sẻ dữ liệu toàn hệ thống.
- Bước 4 - Đánh giá và hoàn thiện: tổng hợp thiết kế bằng công cụ Vivado để đánh giá tài nguyên phần cứng và năng lượng tiêu thụ; xây dựng kịch bản kiểm tra tự động trên FPGA theo đúng luồng hoạt động của một processor thật (đọc chương trình từ thẻ SD, nạp lên DRAM, thực thi, ghi output ngược lại thẻ SD - xem Nội dung 4); xây dựng môi trường verification bằng UVM cho các IP cache/coherence trọng yếu; tổng hợp kết quả và viết báo cáo khóa luận.

Các nội dung chính và giới hạn của đề tài

Nội dung 1: Tìm hiểu về kiến trúc tập lệnh RISC-V RV32IMA, hệ thống đa lõi và cơ chế cache coherence

Nội dung:

- Tìm hiểu kiến trúc tập lệnh RISC-V, đặc biệt là tập lệnh cơ sở RV32I và các mở rộng M (nhân/chia), A (thao tác nguyên tử).
- Tìm hiểu nguyên lý tổ chức hệ thống đa lõi: cache L1 riêng và L2 dùng chung; coherence kiểu snoop-invalidate; MMU/TLB và page table Sv32; kiến trúc bus hai tầng gồm AHB-Lite cho miền core/L1-L2 và AXI4 cho miền L2-DMA-DRAM-ngoại vi; cơ chế bridge giữa hai miền; DMA; và luồng nạp chương trình từ thẻ SD qua SPI lên bộ nhớ chính trước khi thực thi.

Phương pháp:

- Khảo sát nhu cầu ứng dụng thực tế cần bộ xử lý đa lõi, các công trình liên quan, đặc tả RISC-V, AMBA AHB-Lite và AMBA AXI4; so sánh phạm vi chức năng, handshake, arbitration, burst, back-pressure và khả năng tích hợp DMA/DRAM để xác lập vai trò của từng bus trong kiến trúc đề xuất.
- Phân tích ưu, nhược điểm của các phương án tổ chức cache, coherence, quản lý bộ nhớ ảo và giao tiếp bộ nhớ ngoài đã khảo sát, từ đó lựa chọn kiến trúc phù hợp cho đề tài.

Kết quả dự kiến: Nắm vững nguyên lý hoạt động của bộ xử lý RISC-V, tổ chức hệ thống đa lõi, cơ chế coherence, cơ chế quản lý bộ nhớ ảo và cơ chế nạp chương trình từ bộ nhớ ngoài, làm cơ sở cho việc thiết kế chi tiết ở các nội dung tiếp theo.

Nội dung 2: Thiết kế và hiện thực lõi xử lý RV32IMA đơn với cache L1 riêng

Nội dung:

- Thiết kế lõi xử lý RV32IMA theo kiến trúc pipeline 5 tầng (Fetch - Decode - Execute - Memory - Writeback), hỗ trợ tập lệnh cơ sở RV32I cùng mở rộng M và A.
- Thiết kế cache L1 (lệnh/dữ liệu) riêng với bộ điều khiển cache theo thuật toán cấp phát/thay thế set-associative, bộ quản lý bộ nhớ (MMU) cho lõi đơn - gồm TLB 16-entry và bộ dò trang 2 cấp kiểu Sv32, chuyển đổi địa chỉ ảo sang địa chỉ vật lý trước khi truy cập cache/bộ nhớ - và khối giao tiếp giữa lõi xử lý với cache L1 cho một lõi đơn.

Phương pháp:

- Sử dụng ngôn ngữ SystemVerilog để hiện thực từng tầng pipeline, cache L1, bộ điều khiển cache và MMU.
- Xây dựng testbench mô phỏng kiểm tra các nhóm lệnh RV32I, RV32M, RV32A và cơ chế chuyển đổi địa chỉ của MMU trên một lõi đơn, đối chiếu kết quả với mô hình tham chiếu (Spike) ở phạm vi thực thi lệnh (nội dung thanh ghi/bộ nhớ), chưa bao gồm hành vi CSR/trap/đặc quyền.

Kết quả dự kiến: Lõi RV32IMA đơn hoạt động đúng chức năng ở cả ba nhóm lệnh I/M/A, với cache L1 riêng và MMU thực hiện đúng việc chuyển đổi địa chỉ ảo sang vật lý, làm nền tảng ổn định để mở rộng lên hệ thống nhiều lõi.

Nội dung 3: Mở rộng thành hệ thống 4 lõi với AHB-Lite interconnect, AXI4 system bus, cache L2 dùng chung và cơ chế coherence

Nội dung:

- Mở rộng thiết kế lõi đơn thành hệ thống 4 lõi RV32IMA, mỗi lõi có cache L1 và MMU/TLB riêng. Bốn lõi truy cập cache L2 dùng chung qua AHB-Lite interconnect; L2, DMA, DRAM và ngoại vi thuộc miền AXI4 system bus. Giao diện giữa hai miền bảo đảm chuyển đổi giao dịch, back-pressure và thứ tự truy cập nhất quán.
- Hiện thực cơ chế duy trì tính nhất quán bộ nhớ đệm giữa bốn lõi theo giao thức MESI, bao gồm quản lý trạng thái dòng cache, điều phối quyền ghi đọc quyền và xử lý các truy cập đọc/ghi dữ liệu chia sẻ giữa các lõi.

Phương pháp:

- Thiết kế arbiter/interconnect AHB-Lite nhiều master, giao diện AXI4 với các kênh đọc/ghi độc lập và burst transfer, bridge giữa hai miền bus, coherence manager và DMA; xây dựng testbench kiểm tra protocol assertions, timeout, back-pressure và tranh chấp truy cập.
- Xây dựng testbench mô phỏng các kịch bản producer/consumer và truy cập đồng thời giữa các lõi để kiểm chứng tính đúng đắn của dữ liệu chia sẻ.

Kết quả dự kiến: Hệ thống 4 lõi hoạt động ổn định trong mô phỏng, dữ liệu chia sẻ giữa các lõi luôn nhất quán, không xảy ra tình trạng đọc/ghi sai dữ liệu do cache cũ.

Nội dung 4: Đánh giá hệ thống, chạy demo và viết báo cáo

Nội dung:

- Đánh giá độ chính xác chức năng, tài nguyên phần cứng sử dụng (LUT, FF, BRAM,...) và năng lượng tiêu thụ ước tính của thiết kế khi tổng hợp bằng Vivado.

- Xây dựng kịch bản kiểm tra theo 3 mức: (i) mô phỏng mức waveform (waveform-level simulation) chứng minh đúng chức năng logic - mức tối thiểu bắt buộc; (ii) kiểm chứng bằng UVM để đánh giá thiết kế/IP bài bản hơn trước khi tổng hợp; (iii) kiểm tra mức phần cứng (hardware-level) trên FPGA - bắt buộc phải tổng hợp chạy được thật trên board, với kịch bản đánh giá tự động, không thao tác thủ công.

Phương pháp:

- Thiết kế và tích hợp giao tiếp với bộ nhớ ngoài dạng thẻ nhớ (SD card) qua chuẩn SPI, sử dụng IP có sẵn của Vivado để đảm bảo tiến độ, làm kịch bản kiểm tra trên FPGA đúng theo luồng hoạt động của một processor thật, đồng thời dùng làm bước đánh giá hiệu năng/tốc độ của hệ thống quad-core ở giai đoạn kiểm tra cuối cùng: (1) CPU đọc chương trình từ thẻ SD, (2) nạp lên bộ nhớ chính (DRAM) qua bộ điều khiển DMA, (3) CPU thực thi chương trình từ bộ nhớ chính, và (4) khi kết thúc, ghi kết quả xử lý ngược lại thẻ SD - không nạp sẵn chương trình vào BlockRAM của FPGA rồi chạy trực tiếp từ đó.
- Sử dụng công cụ tổng hợp của Vivado để đánh giá tài nguyên sử dụng và ước lượng năng lượng tiêu thụ; nạp thiết kế lên kit AMD Virtex™ UltraScale+™ 56G PAM4 VCU129 FPGA Evaluation Kit để chạy kịch bản kiểm tra tự động theo luồng thẻ SD - DRAM - CPU đã nêu ở trên, đồng thời đo đặc hiệu năng/tốc độ xử lý thực tế của hệ thống quad-core.

Kết quả dự kiến: Hệ thống bộ xử lý 4 lõi RV32IMA hoạt động đúng chức năng ở cả ba mức kiểm tra, đáp ứng yêu cầu về tài nguyên phần cứng hợp lý, có kịch bản kiểm tra trên FPGA phản ánh đúng cách một processor thật hoạt động cùng số liệu hiệu năng/tốc độ thực đo, và có đầy đủ số liệu đánh giá cùng báo cáo khóa luận hoàn chỉnh.

Giới hạn đề tài:

- Đề tài tập trung vào kiến trúc bộ xử lý và hệ thống bộ nhớ đệm nhiều lõi, không mở rộng sang mở rộng dấu chấm động (F/D) hay nén lệnh (C). Bộ quản lý bộ nhớ (MMU) được đưa vào phạm vi chính thức của đề tài, hỗ trợ mô hình phân trang 2 cấp kiểu Sv32 với TLB 16-entry cho mỗi lõi; MMU không hỗ trợ trang lớn (superpage), không tự động cập nhật phần cứng cho bit Accessed/Dirty, và không thực thi phân quyền truy cập theo chế độ đặc quyền (M/S/U) do thiết kế hiện chưa có CSR. Khi có lỗi chuyển đổi địa chỉ (page fault), thiết kế chặn/vô hiệu hóa truy cập tương ứng thay vì phát sinh ngoại lệ tới một bộ xử lý ngoại lệ, vì cơ chế trap/exception chưa nằm trong phạm vi đề tài. Việc đối chiếu với mô hình tham chiếu Spike giới hạn ở kết quả thực thi lệnh I/M/A, không bao gồm hành vi CSR/trap/đặc quyền.
- Số lượng lõi xử lý được giới hạn ở 4 lõi; cơ chế coherence được hiện thực và đánh giá trong phạm vi mô phỏng RTL và tổng hợp trên FPGA, chưa xét đến bài toán mở rộng lên số lượng lõi lớn hơn hoặc hệ điều hành đa nhiệm đầy đủ. Việc kiểm chứng bằng UVM được triển khai cho các IP cache/coherence trọng yếu, bổ sung cho mô phỏng waveform-level, không thay thế yêu cầu bắt buộc về mô phỏng RTL và kiểm tra trên FPGA.

Demo dự kiến:

- Demo bắt buộc (mô phỏng RTL): chạy mô phỏng RTL toàn hệ thống 4 lõi trên Vivado xsim với testbench SystemVerilog, có kịch bản kiểm tra tự động và log kết quả PASS/FAIL cho từng kịch bản (tính toán độc lập trên từng lõi, chia sẻ dữ liệu và đồng bộ hóa giữa các lõi thông qua cơ chế coherence, chuyển đổi địa chỉ qua MMU).

- Verification bằng UVM: xây dựng môi trường kiểm chứng UVM (testbench, sequence, scoreboard, coverage) cho các IP trọng yếu - cache controller (L1/L2) và coherence manager - trước khi tổng hợp, thực hiện song song với giai đoạn thiết kế 4 lõi (xem Kế hoạch thực hiện).
- Demo bắt buộc (phần cứng FPGA): tổng hợp thiết kế và nạp lên kit AMD Virtex™ UltraScale+™ 56G PAM4 VCU129 FPGA Evaluation Kit để chạy thử nghiệm thực tế hệ thống 4 lõi, với chương trình được đọc từ thẻ SD qua giao tiếp SPI (dùng IP có sẵn của Vivado), nạp lên DRAM qua DMA rồi mới thực thi, và kết quả xử lý được ghi ngược lại thẻ SD khi hoàn tất - có kịch bản đánh giá tự động, đồng thời đo đặc hiệu năng/tốc độ xử lý thực tế của hệ thống quad-core, quan sát trạng thái qua giao tiếp UART hoặc LED/GPIO trên kit.

Kế hoạch thực hiện

- Võ Lê Phước Lâm - 23520838: phụ trách chính thiết kế và hiện thực lõi xử lý RV32IMA (pipeline 5 tầng, tập lệnh RV32I/M/A), cache L1 riêng cho từng lõi, bộ điều khiển cache set-associative, bộ quản lý bộ nhớ (MMU/TLB) cho từng lõi, và giao tiếp thẻ SD/SPI (dùng IP Vivado) cùng kịch bản kiểm tra tự động trên FPGA; đồng thời xây dựng môi trường UVM cho khối cache L1 và MMU.
- Vũ Thành Lam - 23520840: phụ trách chính thiết kế interconnect AHB và AXI4 system bus, cache L2 dùng chung, cơ chế duy trì tính nhất quán bộ nhớ đệm (cache coherence) giữa các lõi, hỗ trợ phần MMU/TLB, và bộ điều khiển DMA (khởi được giảng viên hướng dẫn nhấn mạnh là trọng tâm của đề tài); đồng thời xây dựng môi trường UVM cho khối cache L2 và coherence manager.
- Cả hai sinh viên cùng phối hợp khảo sát tài liệu tổng quan, xây dựng testbench, mô phỏng kiểm chứng (bao gồm MMU/TLB), tổng hợp đánh giá tài nguyên/hiệu năng, thực hiện demo và viết báo cáo khóa luận tốt nghiệp.

STT	Nội dung	Tháng 9	Tháng 10	Tháng 11	Tháng 12
1	Tìm hiểu về kiến trúc tập lệnh RISC-V RV32IMA, hệ thống đa lõi và cơ chế cache coherence	x			
2	Thiết kế và hiện thực lõi xử lý RV32IMA đơn với khối giao tiếp bộ nhớ đệm	x	x		
3	Mở rộng thành hệ thống 4 lõi với AHB-Lite, AXI4, cache L2 dùng chung và coherence		x	x	
4	Đánh giá hệ thống và viết báo cáo			x	x

Tuần	Võ Lê Phước Lâm - 23520838	Vũ Thành Lam - 23520840	Mốc
Tuần 1 (07/09-13/09, đã hoàn thành)	Hoàn thiện đề cương chi tiết và kiến trúc tổng thể.	Hoàn thiện address mapping (VA/PA, tag/index/offset cache L1/L2).	Đề cương và address mapping hoàn chỉnh; cả nhóm chốt kiến trúc tổng thể.
Tuần 2 (14/09-20/09)	Hoàn thiện testbench MMU/TLB/PTW: hit, miss, refill, page walk 2 cấp và page	Chốt kiến trúc bus hai tầng; đặc tả AHB-Lite core/L1-L2 và AXI4 L2-DMA-	Đặc tả MMU/TLB,

	fault bằng page table mẫu.	DRAM/ngoại vi; lập ma trận địa chỉ và giao diện bridge.	AHB-Lite và AXI4 được chốt ngay trong tháng 9.
Tuần 3 (21/09-27/09)	Hoàn thiện RTL và regression MMU/TLB/PTW; kiểm tra TLB flush, đồng thời truy cập lệnh/dữ liệu và lỗi dịch địa chỉ.	Hiện thực AHB-Lite arbiter nhiều master và AXI4 read/write channel cơ bản; viết testbench handshake, wait-state và back-pressure.	MMU/TLB PASS kiểm tra đơn vị; AHB-Lite và AXI4 hoàn thành luồng giao dịch cơ bản.
Tuần 4 (28/09-04/10)	Tích hợp MMU/TLB với lõi RV32IMA và cache L1; kiểm tra thứ tự VA - dịch địa chỉ - PA - truy cập cache/bus.	Hoàn thiện AXI4 burst và bridge AHB-Lite/AXI4; bắt đầu DMA mem-to-mem trên miền AXI4.	Mốc cuối tháng 9/đầu tháng 10: MMU/TLB, AHB-Lite, AXI4 và DMA v1 PASS ở mức đơn vị.
Tuần 5 (05/10-11/10)	Hoàn thiện khối lõi đơn RV32IMA + MMU/TLB + Cache L1 + AHB-Lite master; đối chiếu I/M/A với Spike.	Tích hợp AHB-Lite interconnect với mô hình L2 và AXI4 system bus; kiểm tra chuyển tiếp giao dịch qua bridge.	Khối lõi đơn và đường truy cập xuyên AHB-Lite/AXI4 PASS.
Tuần 6 (12/10-18/10)	Nhân bản khối lõi đơn đã có MMU/TLB và Cache L1 thành 4 lõi; đấu nối các AHB-Lite master.	Hoàn thiện phân xử AHB-Lite 4 lõi, kiểm tra fairness/timeout; ổn định AXI4 system bus cho DMA và DRAM model.	Mốc giữa tháng 10: bốn lõi truy cập qua AHB-Lite và đi đến miền AXI4 đúng giao thức.
Tuần 7 (19/10-25/10)	Hỗ trợ debug tích hợp MMU/TLB, cache L1 và giao tiếp AHB-Lite trên bốn lõi.	Thiết kế và RTL cache L2 dùng chung; nối L2 với AHB-Lite phía lõi và AXI4 phía hệ thống.	Cache L2 có thể nhận giao dịch từ bốn lõi và phát giao dịch xuống AXI4.
Tuần 8 (26/10-01/11)	Xây dựng regression/UVM ưu tiên cho MMU/TLB và AHB-Lite; khóa các lỗi giao tiếp trước khi mở rộng coherence.	Tích hợp L2; bắt đầu coherence và bổ sung assertion/coverage cho AXI4 cùng đường bridge.	Mốc cuối tháng 10: nền tảng MMU/TLB + AHB-Lite + AXI4 ổn định để triển khai coherence và DMA nâng cao.
Tuần 9 (02/11-08/11)	Tiếp tục UVM cache L1/MMU.	Mở rộng MESI đủ 4 trạng thái (Modified/Exclusive/Shared/Invalid), quản lý quyền ghi đọc quyền.	
Tuần 10 (09/11-15/11)	Cùng Lam xây dựng testbench producer/consumer trên 4 lõi.	Xây dựng UVM cho Cache L2 và coherence manager; testbench producer/consumer trên 4 lõi.	Nội dung 3 hoàn tất, coherence PASS mô phỏng.
Tuần 11 (16/11-22/11)	Thiết kế bộ điều khiển SD card/SPI (dùng IP Vivado), chuẩn bị đấu vào AXI4 system bus.	Hoàn thiện DMA đầy đủ - nối DMA vào AXI4 system bus thật, hỗ trợ transfer với nguồn ngoài.	Tuần trọng tâm của khối DMA theo yêu cầu của giảng viên hướng dẫn.

Tuần 12 (23/11-29/11)	Ghép SD/SPI controller - DMA – AXI4 system bus, test đọc thử một block từ mô hình SD giả lập.	Hỗ trợ debug đường DMA-SD, hoàn thiện UVM còn lại.	Đường SD - DMA - DRAM chạy được trong mô phỏng.
Tuần 13 (30/11-06/12)	Tích hợp toàn hệ thống lần 1 (4 lõi + Cache L1/L2 + MESI + AHB + AXI4 + DMA + SD/SPI), dành trọn tuần để debug tích hợp, không thêm tính năng mới.	Tích hợp toàn hệ thống lần 1 (cùng Lâm), dành trọn tuần để debug tích hợp, không thêm tính năng mới.	Tuần rủi ro tiến độ cao nhất.
Tuần 14 (07/12-13/12)	Fix lỗi tích hợp, hoàn thiện testbench toàn hệ thống.	Fix lỗi tích hợp, hoàn thiện testbench toàn hệ thống (cùng Lâm).	Demo mô phỏng RTL bắt buộc PASS.
Tuần 15 (14/12-20/12)	Tổng hợp thiết kế bằng Vivado (synthesis), đánh giá tài nguyên LUT/FF/BRAM.	Tổng hợp thiết kế bằng Vivado (cùng Lâm).	
Tuần 16 (21/12-27/12)	Bắt đầu bring-up FPGA thật trên kit AMD Virtex UltraScale+ VCU129 (clock, reset, I/O, khe SD vật lý).	Bring-up FPGA (cùng Lâm).	
Tuần 17 (28/12-03/01)	Debug bring-up FPGA.	Debug bring-up FPGA (cùng Lâm) - giai đoạn thường phát sinh khác biệt so với mô phỏng, cần chờ đủ thời gian.	
Tuần 18 (04/01-10/01)	Chạy kịch bản tự động trên FPGA thật: SD - DMA - DRAM - CPU thực thi - ghi ngược SD; đo hiệu năng/tốc độ quad-core thực tế.	Cùng chạy kịch bản tự động và đo hiệu năng (cùng Lâm).	Demo phần cứng FPGA bắt buộc PASS.
Tuần 19 (11/01-17/01)	Dự phòng - fix lỗi FPGA nếu tuần 18 chưa đạt, hoặc hoàn thiện UVM còn thiếu.	Dự phòng (cùng Lâm).	Không xếp công việc mới bắt buộc vào tuần này.
Tuần 20 (18/01-24/01)	Viết Báo cáo chi tiết/tóm tắt, làm slide, dựng video demo, rà soát nội dung khớp thực tế đã làm.	Viết báo cáo, slide, video demo (cùng Lâm).	Nộp báo cáo trước hoặc đúng ngày 25/01/2027.

TÀI LIỆU THAM KHẢO

- [1] M. Solé *et al.*, "Evaluation of the Multicore Performance Capabilities of the Next Generation Flight Computers," trong *Proc. 2023 IEEE/AIAA 42nd Digital Avionics Systems Conference (DASC)*, 2023, doi: 10.1109/DASC58513.2023.10311151.
- [2] B. Kieu-Do-Nguyen *et al.*, "Hardware Software Co-design for Multi-threaded Computation on RISC-V-based Multicore System," *IEEE Access*, vol. 12, pp. 177312-177326, 2024, doi: 10.1109/ACCESS.2024.3505940.
- [3] S. Shukla và K. C. Ray, "A Low-Overhead Reconfigurable RISC-V Quad-Core Processor Architecture for Fault-Tolerant Applications," *IEEE Access*, vol. 10, pp. 44136-44146, 2022, doi: 10.1109/ACCESS.2022.3168864.
- [4] N. Đ. T. Tuệ, "Thiết kế bộ vi xử lý 2 lõi RISC-V dựa trên kiến trúc tập lệnh RV32IMFA," *Khóa luận tốt nghiệp*, Khoa Kỹ thuật Máy tính, Trường Đại học Công nghệ Thông tin, ĐHQG TP.HCM, 2026.
- [5] V. C. G. Luật và L. M. Hùng, "Thiết kế bộ điều khiển đồng bộ cache cho các hệ thống 2 lõi dựa trên kiến trúc tập lệnh RISC-V RV32IA," *Khóa luận tốt nghiệp*, Khoa Kỹ thuật Máy tính, Trường Đại học Công nghệ Thông tin, ĐHQG TP.HCM, 2026.

Xác nhận của CBHD

(Ký tên và ghi rõ họ tên)



TS. Nguyễn Minh Sơn

TP. HCM, ngày 10 tháng 09 năm 2026

Sinh viên

(Ký tên và ghi rõ họ tên)



Võ Lê Phước Lâm

Vũ Thành Lâm